

4. Data Visualization I

1. Use the inbuilt dataset 'titanic'. The dataset contains 891 rows and contains information about the passengers who boarded the unfortunate Titanic ship. Use the Seaborn library to see if we can find any patterns in the data.
2. Write a code to check how the price of the ticket (column name: 'fare') for each passenger is distributed by plotting a histogram.

In [1]:

```
import numpy as np
import pandas as pd
import seaborn as sns
```

4.1. Preprocessing

4.1.1. Structure of the dataset

```
survival      Survival      0 = No, 1 = Yes
pclass        Ticket class   1 = 1st, 2 = 2nd, 3 = 3rd
sex           Sex
Age           Age in years
sibsp         # of siblings / spouses aboard the Titanic
parch         # of parents / children aboard the Titanic
ticket        Ticket number
fare          Passenger fare
cabin         Cabin number
embarked       Port of Embarkation C = Cherbourg, Q = Queenstown, S = Southampton
```

In [2]:

```
ds = pd.read_csv( "titanic.csv" )
print( ds.shape )
ds.describe()
```

(891, 12)

Out[2]:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

In [3]:

```
ds.dtypes
```

Out[3]:

Out[3]:

```
PassengerId      int64
Survived          int64
Pclass           int64
Name             object
Sex              object
Age             float64
SibSp            int64
Parch            int64
Ticket           object
Fare            float64
Cabin           object
Embarked         object
dtype: object
```

4.1.2. Correcting data-types (optional)

In [4]:

```
ds = ds.astype( {
    "Name": "string" ,
    "Sex": "string" ,
    "Ticket": "string" ,
    "Cabin": "string" ,
    "Embarked": "string" ,
} )
ds.dtypes
```

Out[4]:

```
PassengerId      int64
Survived          int64
Pclass           int64
Name             string[python]
Sex              string[python]
Age             float64
SibSp            int64
Parch            int64
Ticket           string[python]
Fare            float64
Cabin           string[python]
Embarked         string[python]
dtype: object
```

4.1.3. Handling missing values

In [5]:

```
# Check number of missing values
ds.isna().sum()
```

Out[5]:

```
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age             177
SibSp            0
Parch            0
Ticket           0
Fare            0
Cabin           687
Embarked         2
dtype: int64
```

In [6]:

```
# Check statistics of Age
```

```
# Check statistics of Age
print( ds[ "Age" ].describe() )

# As evident from the small std-dev of Age, most passengers
# were young individuals, as indicated from a mean of 29 and median
# of 28 years.
# Replace missing ages with their median
ds["Age"] = ds[ "Age" ].fillna( ds[ "Age" ].median() )
```

```
count      714.000000
mean       29.699118
std        14.526497
min         0.420000
25%        20.125000
50%        28.000000
75%        38.000000
max        80.000000
Name: Age, dtype: float64
```

In [7]:

```
# Cabin values are (near) unique to each passenger
# Hence, summary statistics cannot be used for imputation
# Moreover, the feature is never used for data analysis,
# as its value is (near) unique for each passenger
# Hence, we skip its operation
```

4.2. Exploratory Data Analysis

4.2.1. Visualization of each feature with target class Survived

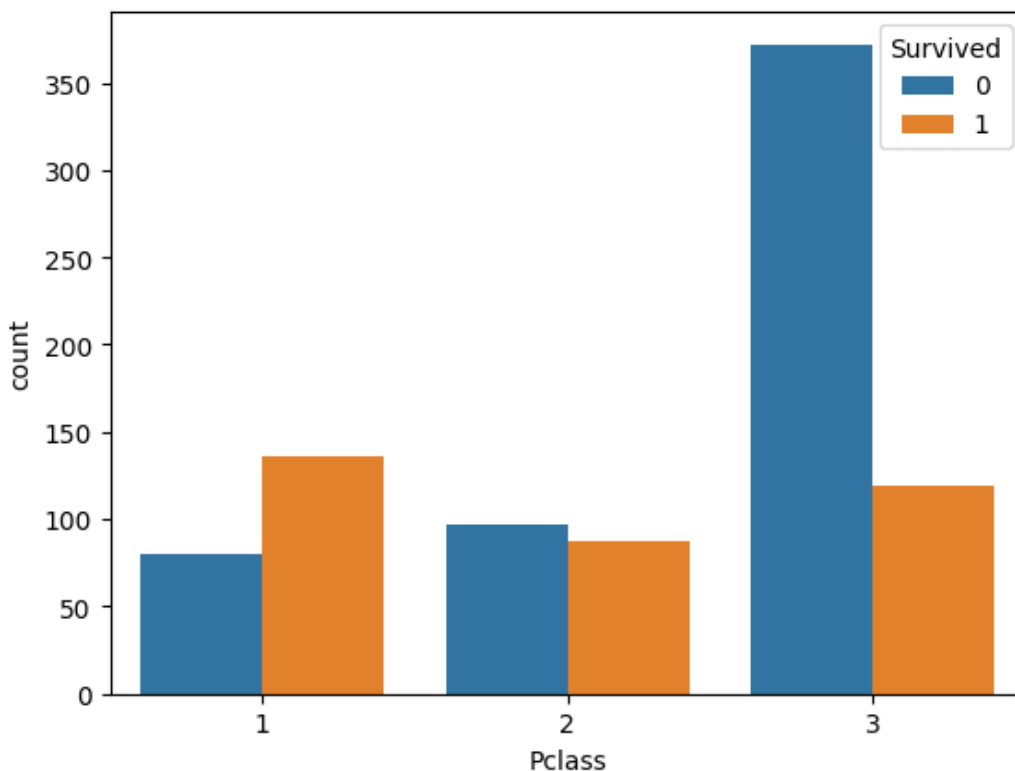
Observation: Pclass = 3 (third class) passengers survived the least and Pclass = 1 (first class) passengers survived the most

In [8]:

```
# PClass vs Survived
sns.countplot( data=ds , x="Pclass" , hue="Survived" )
```

Out[8]:

<Axes: xlabel='Pclass', ylabel='count'>



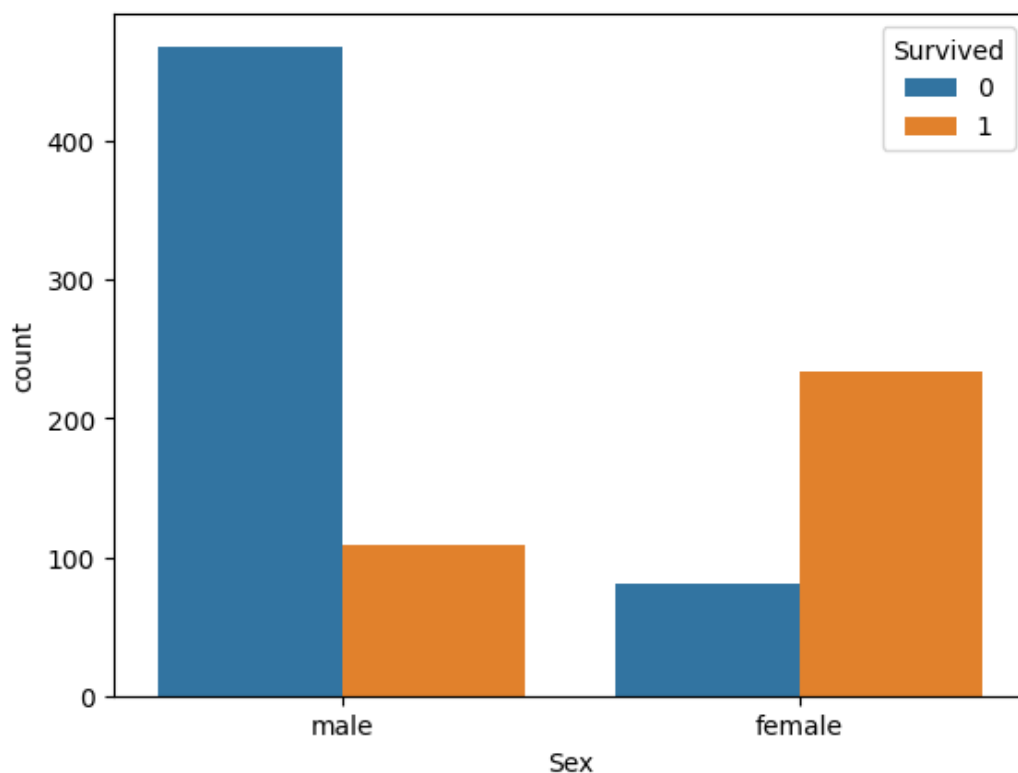
Observation: Males survived least, females the most

In [21]:

```
# Sex vs. Survived
sns.countplot( data=ds , x="Sex" , hue="Survived" )
```

Out[21]:

<Axes: xlabel='Sex', ylabel='count'>



Observation: The distributions of Age for passengers surviving or not, are identical, considering their summary statistics below

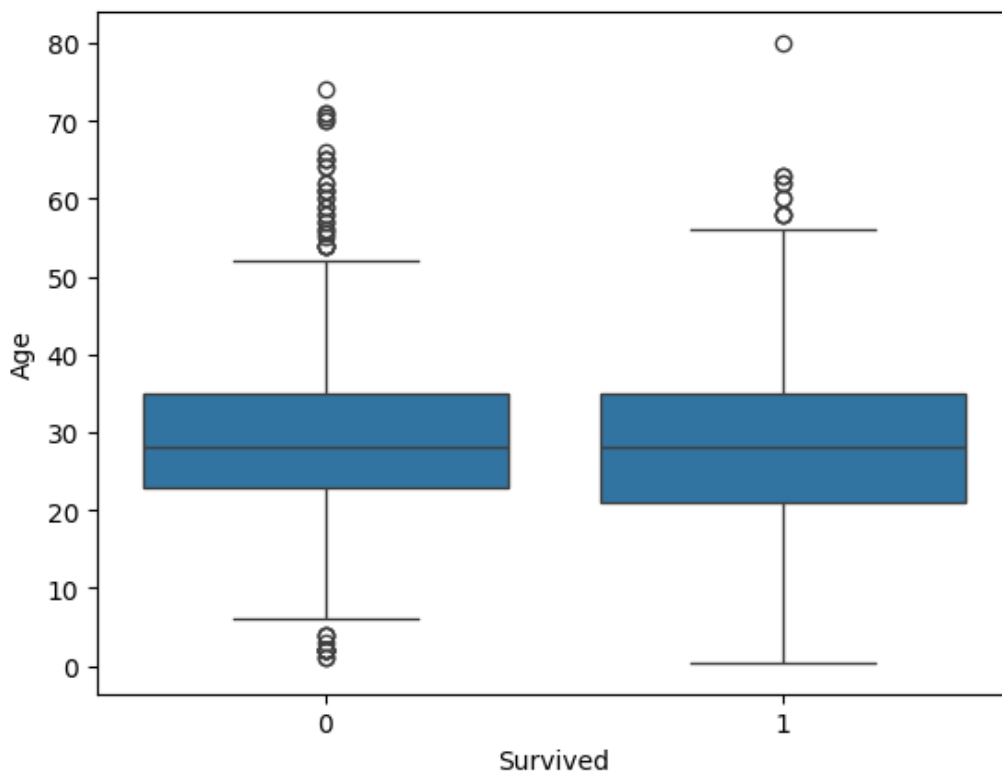
In [12]:

```
# Age vs. Survived
print( ds.loc[ ds["Survived"] == 0 , "Age" ].describe() )
print( ds.loc[ ds["Survived"] == 1 , "Age" ].describe() )
sns.boxplot( data=ds , x="Survived" , y="Age" )
```

```
count    549.000000
mean      30.028233
std       12.499986
min        1.000000
25%       23.000000
50%       28.000000
75%       35.000000
max       74.000000
Name: Age, dtype: float64
count    342.000000
mean      28.291433
std       13.764425
min        0.420000
25%       21.000000
50%       28.000000
75%       35.000000
max       80.000000
Name: Age, dtype: float64
```

Out[12]:

<Axes: xlabel='Survived', ylabel='Age'>



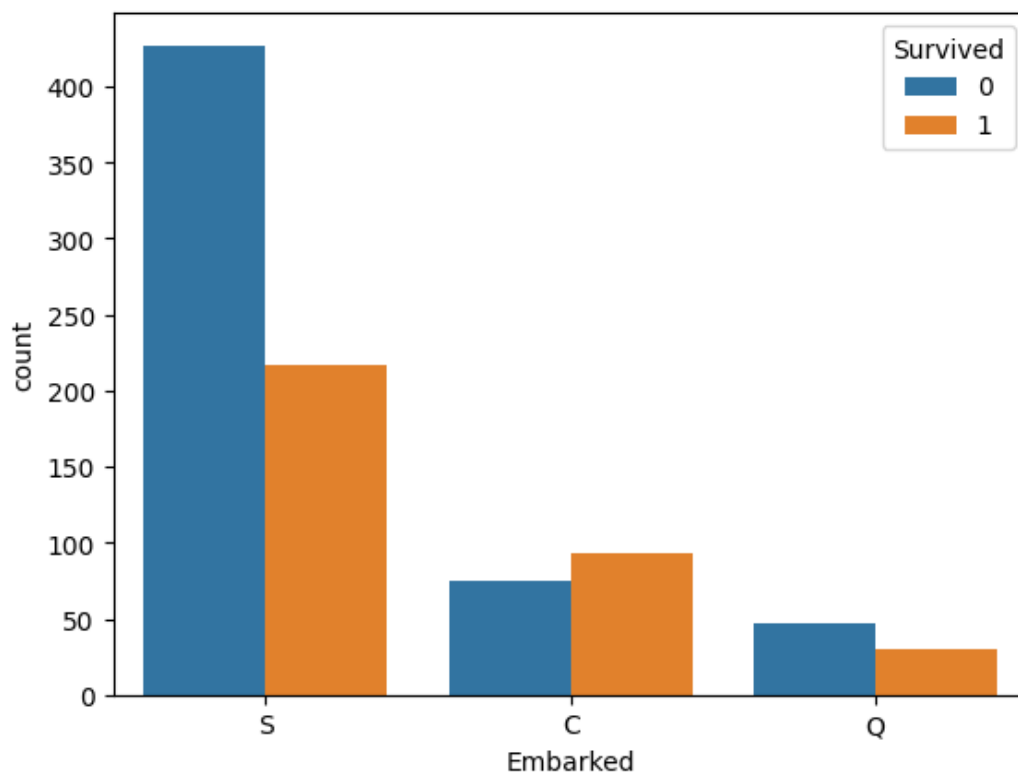
Observation: Most passengers which survived or didn't survive embarked the Titanic from Southampton

In [23]:

```
# Embarked vs. Survived
sns.countplot( data=ds , x="Embarked" , hue="Survived" )
```

Out[23]:

<Axes: xlabel='Embarked', ylabel='count'>



Observation: As first class passengers survived the most, passengers with higher fare survived, assuming the fact that first-class had a higher fare than other classes

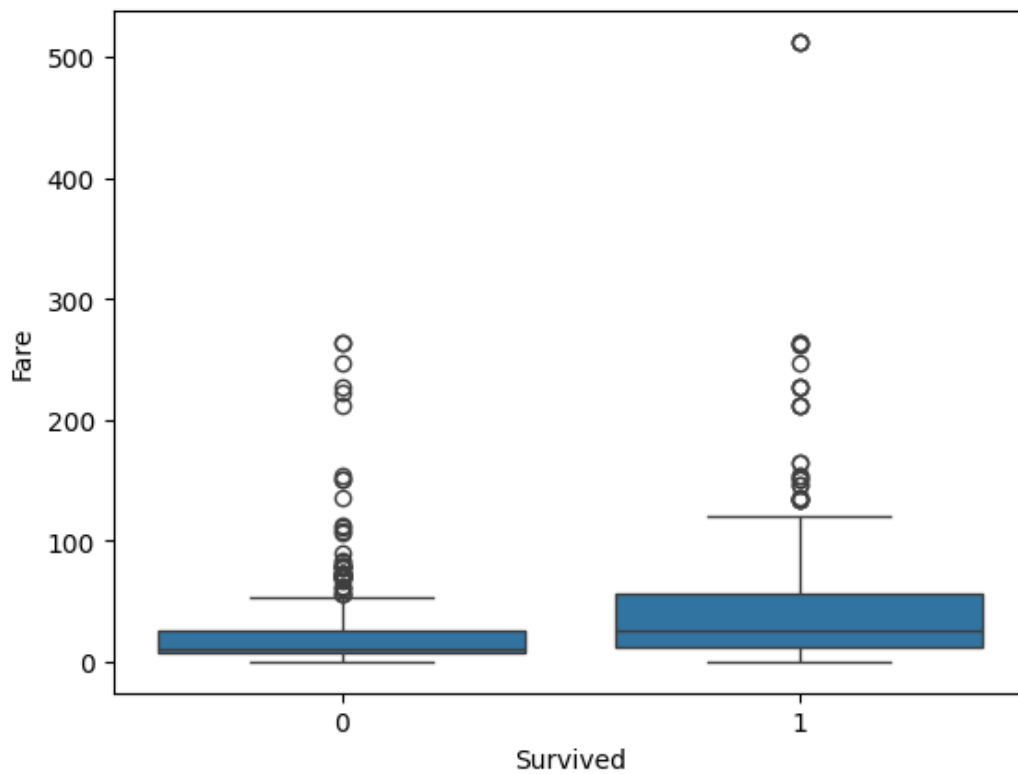
In [24]:

```
# Fare vs. Survived
```

```
sns.boxplot( data=ds , x="Survived" , y="Fare" )
```

Out[24]:

<Axes: xlabel='Survived', ylabel='Fare'>

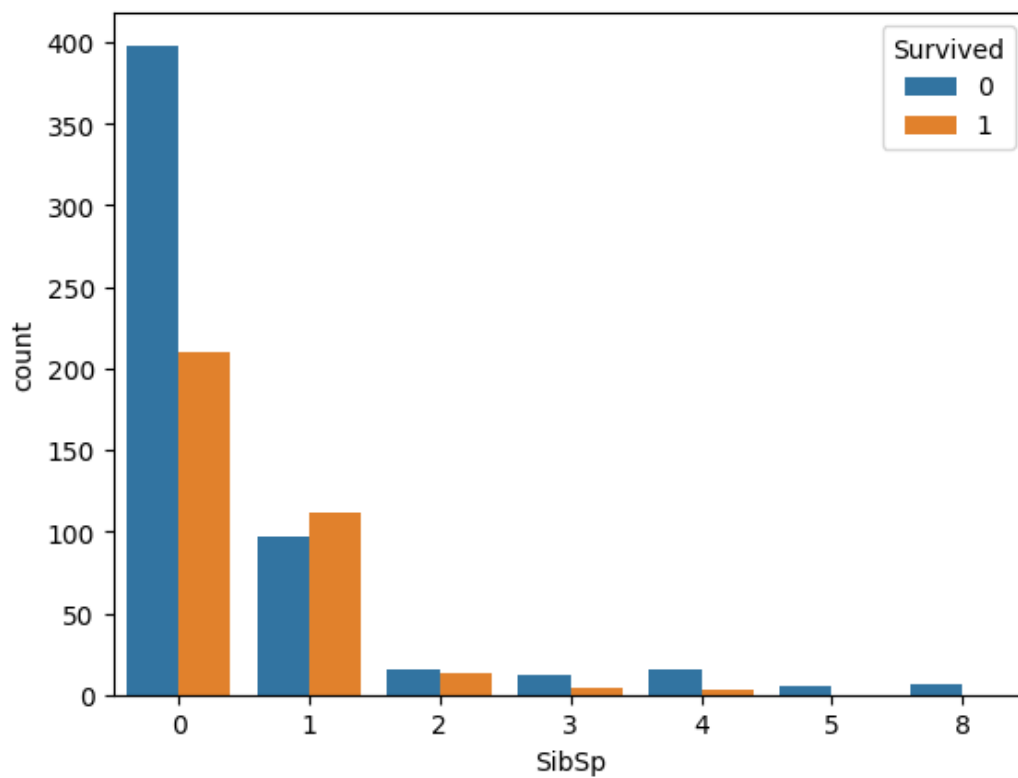


In [25]:

```
# SibSp vs. Survived  
# Most passengers did not have a sibling or spouse on-board  
sns.countplot( data=ds , x="SibSp" , hue="Survived" )
```

Out[25]:

<Axes: xlabel='SibSp', ylabel='count'>



In [18]:

```
# Parch vs. Survived
```

```
# Most passengers did not have a parent or child on-board  
sns.countplot( data=ds , x="Parch" , hue="Survived" )
```

Out[18]:

<Axes: xlabel='Parch', ylabel='count'>

